

# GRAND CHALLENGE 2026

## *Term 1 — Python: Language & Application*

Weeks 9–11 • 6 June – 20 June 2026

Submission Deadline: Thursday 19 June 2026, 23:59

---

### 1. Overview

**Congratulations — you've made it to the final stretch.** Over the past eight weeks you've learned how to write Python from scratch: variables, loops, conditionals, functions, data structures, File I/O, and Object-Oriented Programming. Now it's time to put it all together and build something you're genuinely proud of.

The Grand Challenge runs over three Saturdays: brainstorming, building, and presenting. The best project wins a prize. There's also a **Peer Choice Award** — voted by your fellow students — so pay close attention during every presentation.

**Group size:** Work in groups of 2–4 students. We recommend 3–4 for a more balanced workload.

---

### 2. What to Build

Build whatever you like — in Python — as long as it draws on concepts from the course. You have complete creative freedom.

**We have prepared some prompts below:**

#### Contextual Prompts

- Dr. Jennifer Eberhardt (Psychology, Stanford) studies the impacts of unconscious bias and how stereotypes affect the performance of ourselves and others. In her groundbreaking research, Dr. Eberhardt demonstrates that implicit bias is a societal and systemic issue triggered automatically by our brains to categorize the world. From algorithmic bias in AI to the subtle cues in digital marketplaces and workplace hiring, these unconscious associations reinforce racial disparities and hold people back from reaching their full potential. As technology increasingly mediates our

daily lives, we have a unique opportunity: *How can we better design our tools and technologies to interrupt bias before it leads to unequal outcomes?*

- Dr. Teo You Yenn (Sociology, NTU) studies the impacts and manifestations of inequality of all kinds in Singapore. One such kind is educational inequality. This is the phenomenon where educational outcomes and experiences are heavily shaped by a student's socioeconomic background, rather than just their innate talent or effort. In her influential book *This Is What Inequality Looks Like*, Dr. Teo challenges traditional narratives of meritocracy and inequality in Singapore, showing how different starting lines create a compounding advantage for some while leaving others struggling to keep up. (BbB was formed to address this challenge in coding education.) *How then, can we better support disadvantaged youth in this system, using our tools and technologies to level the playing field?*

#### Open Prompts

- **Social Impact:** Think of a problem in your school, community, or Singapore. Can code help? This is the hardest category — but if you pull it off, it will stand out.
  - **Productivity Tool:** A task you do every day — can you make it faster, easier, or more organised?
  - **Games:** Wordle, Hangman, Tic-Tac-Toe, a text adventure, a puzzle solver. Bonus points if you add your own twist to the game or create an 'AI opponent' who never loses.
  - **Science & Math:** Simulations, password generators, calculators, file encryptors, quiz systems.
- The Goal:** Don't just write code — solve a problem or create an experience. Aim for an MVP (Minimum Viable Product) that is polished, well-documented, and crash-resistant.
- 

### 3. Project Timeline

| Week    | Date        | Focus                                    |
|---------|-------------|------------------------------------------|
| Week 9  | Sat 6 June  | Kickoff, Group Formation & Brainstorming |
| Week 10 | Sat 13 June | Build Phase & Tutor Desk Reviews         |
| Week 11 | Sat 20 June | Pitch Day — Presentations & Awards       |

#### Week 9 — Saturday 6 June: Kickoff & Brainstorming

This is your planning day. By the end of the session, every group should have a locked-in idea and a clear sense of who owns what. You'll also have full access to the judges + experts to bounce ideas off them, ask questions and perfect your pitch!

**What happens during the session:**

- Groups formed (2–4 students per group)
- Facilitated brainstorm: pitch your idea, get feedback from tutors and peers, refine it
- Ask the judges/experts questions (remember, some of the knowledge you learn from them will benefit you beyond the class!)
- Create the GitHub repository where you'll host your code!
- Draft your README title and feature list before the session ends

**Before next session:** Sketch your code structure. Decide who owns which functions or classes. No coding blind spots.

## **Week 10 — Saturday 13 June: Build Phase & Tutor Review**

Come with working code. Even a skeleton is fine — but blank files are not. You have one week left after today.

**What happens during the session:**

- Tutor desk reviews: every group gets structured feedback on code quality, scope, and logic from the tutors
- Coding time with tutors available throughout for debugging and advice
- Test your project — if it breaks there, fix it now

**Before submission:** Polish your code, complete your README, and test one final time. Submission opens after this session.

## **Week 11 — Saturday 20 June: Pitch Day**

Your product launch. See Section 5 for the full Pitch Day format and judge details.

---

## 4. Submission Requirements

**Deadline: Thursday 19 June 2026, 23:59**

Submit via GitHub (See [Annex A](#)). A complete submission includes all three of the following:

### A. Your Code

- Must be written in Python
- Must include a `main.py` file as the entry point of your program
- Should be modular — use functions and classes where appropriate
- Must handle errors gracefully: unhandled crashes on bad user input

### B. Your README.md

Include a `README.md` file in your project. This is the front door of your project — make it count.

1. **Title & Description:** What is this project? Why did you build it? (2–3 sentences)
2. **Features:** A bulleted list of the main functionalities. E.g. *'Saves high scores to a file using File I/O'* or *'Uses recursion to solve the puzzle.'*
3. **How to Run:** The exact steps to run your project. What needs to be installed? What command starts the program? (`python main.py`)
4. **File Structure:** A simple map of your project files and what each one does.
5. **Reflection:** What was the hardest problem you faced, and how did you solve it? If you had two more weeks, what would you add?

### C. How to Submit

1. Copy your GitHub repo link
2. Make sure your repo is public!
3. Submit your link via the Google Form by **19 June, 23:59**

**Note:** Git was not part of this term's curriculum; Using good version management and naming conventions, however, will benefit you everywhere :)

---

## 5. Pitch Day — Saturday 20 June

Think of this as a product launch. You are not just showing code — you are selling an idea.

### Your Judges

- Mr Jing An Tew, CTO, Stick'em
- Mr Chris Soh, Senior Lecturer in Entrepreneurship, Republic Polytechnic
- Mr Zhen Xian Kee, Principal Trainer & Main Organiser of NCO at NUS
- Mr Yaojie Xiao, Co-Founder of Dibs

### Format — 15 Minutes Per Group

| Segment        | Time  | What to Cover                                                                                       |
|----------------|-------|-----------------------------------------------------------------------------------------------------|
| Pitch          | 3 min | What problem does this solve? Who is it for?                                                        |
| Demo           | 7 min | Run your code live. Show the happy path AND how it handles bad input. This is the star of the show. |
| Code Spotlight | 3 min | Pick one piece of code you're proud of. Walk the judges through how it works.                       |
| Q&A            | 2 min | Answer questions from judges and peers.                                                             |

### Awards

- 🏆 **Best Project** — Awarded to the group with the highest combined judge score across all five rubric categories. The winning team takes home a small prize!
- ✨ **Peer Choice Award** — Voted by your fellow students at the end of all presentations. One vote per person; you cannot vote for your own group. The group with the most votes wins. This group will also get a small prize.

### Tips for a Great Demo

- Practice your demo at least twice before the day. Muscle memory matters under pressure.
- Have a backup plan if the internet fails — a recorded video or screenshots of your output.
- Show what happens when users type the wrong thing. Judges notice error handling.
- The demo is 7 minutes — the longest segment. Don't rush it. This is where you win.

---

## 6. How You'll Be Evaluated

We aren't just looking for '*code that works*.' We're looking for code that is readable, structured, documented, and thoughtful. The rubric below reflects exactly what the judges will score.

| Category                       | Emerging (1–2 pts)                                          | Proficient (3–4 pts)                                                        | Advanced (5 pts)                                                                          | Score |
|--------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------|
| <b>Logic &amp; Syntax</b>      | Code has frequent errors; logic is hard to follow.          | Code runs correctly; uses loops, conditionals, and functions appropriately. | Code is clean and optimised; demonstrates advanced logic (e.g. recursion).                | / 5   |
| <b>Code Structure</b>          | Single long file; no clear organisation ('spaghetti code'). | Uses functions and clear variable names; logical, readable flow.            | Good use of Classes and OOP; code is modular and reusable.                                | / 5   |
| <b>Documentation</b>           | Minimal or missing README; no setup instructions.           | Clear README with title, features, and setup steps.                         | Detailed README with file map, usage examples, and thoughtful reflection.                 | / 5   |
| <b>Data Handling</b>           | No meaningful use of lists, dicts, or File I/O.             | Effectively uses Lists/Dicts or File I/O to manage data.                    | Creative use of data structures and File I/O (e.g. persistent state, structured records). | / 5   |
| <b>Innovation &amp; Effort</b> | Direct copy of a class example with minimal changes.        | Clear original idea; includes features beyond the minimum.                  | Exceptional polish; solves a real problem creatively; handles user error gracefully.      | / 5   |

|             |  |  |  |      |
|-------------|--|--|--|------|
| Total Score |  |  |  | / 25 |
|-------------|--|--|--|------|

---

## 7. Pro Tips for Success

- **Start with a plan, not with code.** Spend the first 30 minutes of Week 9 sketching on paper — what does your program do, what functions does it need, what data does it store? Groups that plan well build faster and debug less.
- **Commit to a small idea and execute it brilliantly.** A polished quiz game beats a half-finished social network every time. Judges reward completion, error handling, clear thought process, and clean code far more than ambitious scope with rough edges.
- **Handle bad input everywhere.** What happens if the user presses Enter on an empty field? Types a letter where a number is expected? Test every input. Robust error handling is one of the clearest signals of a strong coder.
- **Write your README as you build, not after.** It's much easier to document a feature the moment you finish it than to reconstruct it at midnight on deadline day. A great README can move you from Proficient to Advanced in the Documentation category.
- **Use your tutors during Week 10.** The desk review session exists for a reason — bring your toughest bug, your messiest function, or your biggest design question. You won't get this level of free feedback after today.
- **Rehearse the demo out loud, not just in your head.** Saying "and now I'll show you..." while typing feels different from imagining it. Run through the full 7-minute demo at least twice with your group before Pitch Day, including what each person says.

---

*Good luck. Build something you're proud of.*

**Bit by Bit Coding · [bbbcoding.org](https://bbbcoding.org) · @bbbcoding**

## Annex A

GitHub is a cloud-based platform and social network where developers store, share, and collaborate on software code.

Be sure to add your teammates as collaborators on your GitHub repository so they can work on the code together. In your repository, go to Settings > Collaborators. Verify your email to start adding your teammates.

To write your code, we recommend either using GitHub Codespaces or Visual Studio Code as your code editor:

- GitHub Codespaces is a cloud-hosted development environment that allows you to write, run, and debug code entirely in the cloud. You can access your workspace from your web browser using a web version of Visual Studio Code.
- Visual Studio Code (often called VS Code) is a free, lightweight, and highly customizable code editor developed by Microsoft. *Note that you would need a laptop to install VS Code and you may have to spend some time setting it up.*

### [GitHub Codespaces Guide](#)

## Version Control

Version control helps you track every change, lets multiple people work on the same project simultaneously without overwriting each other's work, and allows you to safely roll back to a working version if something breaks.

For the purposes of this project, you can use the Live Share extension for VS Code. This allows you to collaborate in real-time on the same codebase (like Google Docs) without needing to clone repositories or set up local environments.

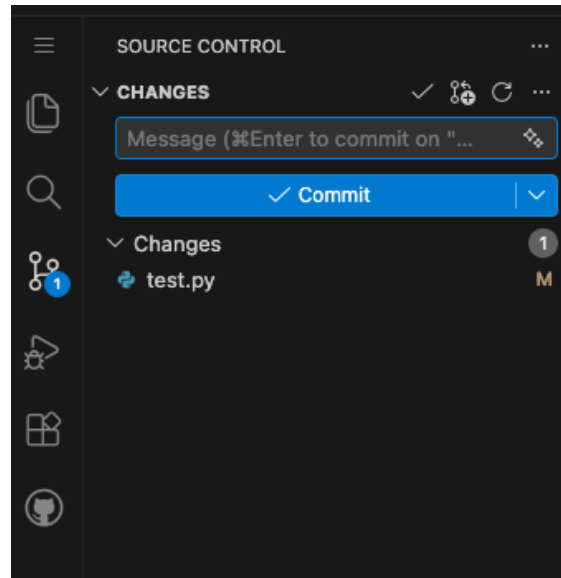
### [Live Share Guide](#)

Before you start a coding session, ensure that you enter the command “git pull” in your terminal before you start coding. This will download and integrate changes that your teammates have made while you were away. **This is very important. If you start coding without doing git pull, you may end up editing an old version of a file, leading to merge conflicts and rejected pushes when you try to share your work later.**

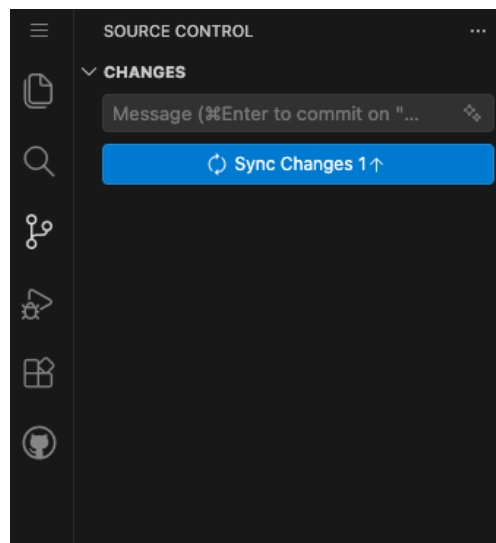
At the end of a coding session, ensure you save your work:



1. Commit your changes (this is like creating a save point in a game) by typing a commit message and clicking on “Commit”. The message should describe briefly what changes you just made. You will be asked whether you would like to stage all your changes and commit them directly - click “Yes”.



2. Now that you have created your “save point”, push your changes to GitHub by clicking on “Sync Changes”. There will be a prompt: This action will pull and push commits from and to "origin/main". Click “OK”



3. Now, you can see the changes in your repository in your GitHub account.